

The “and” test: if you can’t describe a function in one sentence without the word “and”, split it. A function that is hard to name usually has a **design** problem, not a vocabulary problem.

FUNCTIONS

Rule	✓ Do	✗ Don’t
☐ One job — no “and” in the name	<code>validate_student()</code>	<code>validate_and_save()</code>
☐ Specific, not vague	<code>remove_duplicate_emails()</code>	<code>process(data)</code>
☐ Name the intent, not the algorithm	<code>rank_scores()</code>	<code>bubble_sort_scores()</code>
☐ Honest: <code>get_</code> never changes state	<code>charge_fee()</code>	<code>get_balance()</code> # deducts
☐ Booleans ask a question	<code>is_valid()</code> , <code>has_passed()</code>	<code>check(s)</code>
☐ Grammar fits the return value	<code>send_</code> , <code>to_</code> , <code>find_</code>	<code>do_it()</code>
☐ No True/False flag parameters	<code>print_summary()</code>	<code>print_report(s, True)</code>
☐ A long name means split it	<code>read_rows()</code> , <code>average()</code>	<code>read_parse_and_average()</code>
☐ One word per concept	<code>get_</code> everywhere	<code>get_/fetch_/retrieve_</code>

----- style

STYLE (PEP 8)

Rule	✓ Do	✗ Don’t
☐ snake_case for functions and variables	<code>calculate_grade</code>	<code>calculateGrade</code>
☐ UPPER_CASE constants, PascalCase classes	<code>MAX_TRIES</code> , <code>Student</code>	<code>maxTries</code> , <code>student</code>
☐ Leading <code>_</code> for internal helpers	<code>_format_row()</code>	<code>helper2()</code>

Grammar quick key

Action → verb phrase: `send_reminder()`
 True/False → `is_`, `has_`, `can_`
 Value → noun: `average(marks)`
 Conversion → `to_celsius()`
 Might find nothing → `find_student()` (may return `None`)

Before you submit

Read each function name aloud. Does it say **exactly** what the function does — and nothing it hides?
 Read each `if` aloud. Does it sound like English?
 If renaming makes you want to split the function, **do it**.